ments and application logic and even allows server side rendering with a bit of tweaking.

In a sea of frameworks, with a whale in each domain, Angular would be the equivalent of a killer whale, just adept enough to get ALL the jobs done.

Angular vs Better Angular?

There has been a lot of hype ever since the launch of Angular 2.0 back in March 2016 and everybody (us included!) is raving about how google took one of the most popular frameworks on the web and made it better!

Component-Based

Angular is now completely component-based. There are no such things as controllers or $scope. They have been replaced by components and directives. Components are directives with a template.

All of the components that are being used must be made known via bootstrap. They also have to be imported on the page.

Dependency Injection

Dependency Injection has always been one of the key features in Angular but with an improved DI model, there are now more opportunities for component / object-based work. The dependency injection consists of three parts. The Injector, which has the APIs to inject the dependencies and make dependency injection available. Then there are Bindings which make it possible for dependencies to be named, and finally the actual dependencies of the object are generated so that they can be injected.

In Angular 1.x an object is passed into the constructor of the component, but henceforth, this only needs to be passed through the injector view, as shown in the given listing. The objects that are enclosed in the square brackets can then be used to inject.

```
@Component({
      selector:'my-app',
      viewInjector:[MessageList,httpInjectables]
```

Some extra annotations have also been added which enhance the possibilities of dependency injection, for example the @InjectPromise annotation.

With this annotation asynchronicity can be achieved, and the object is only injected when it actually has been created. At the moment of injection